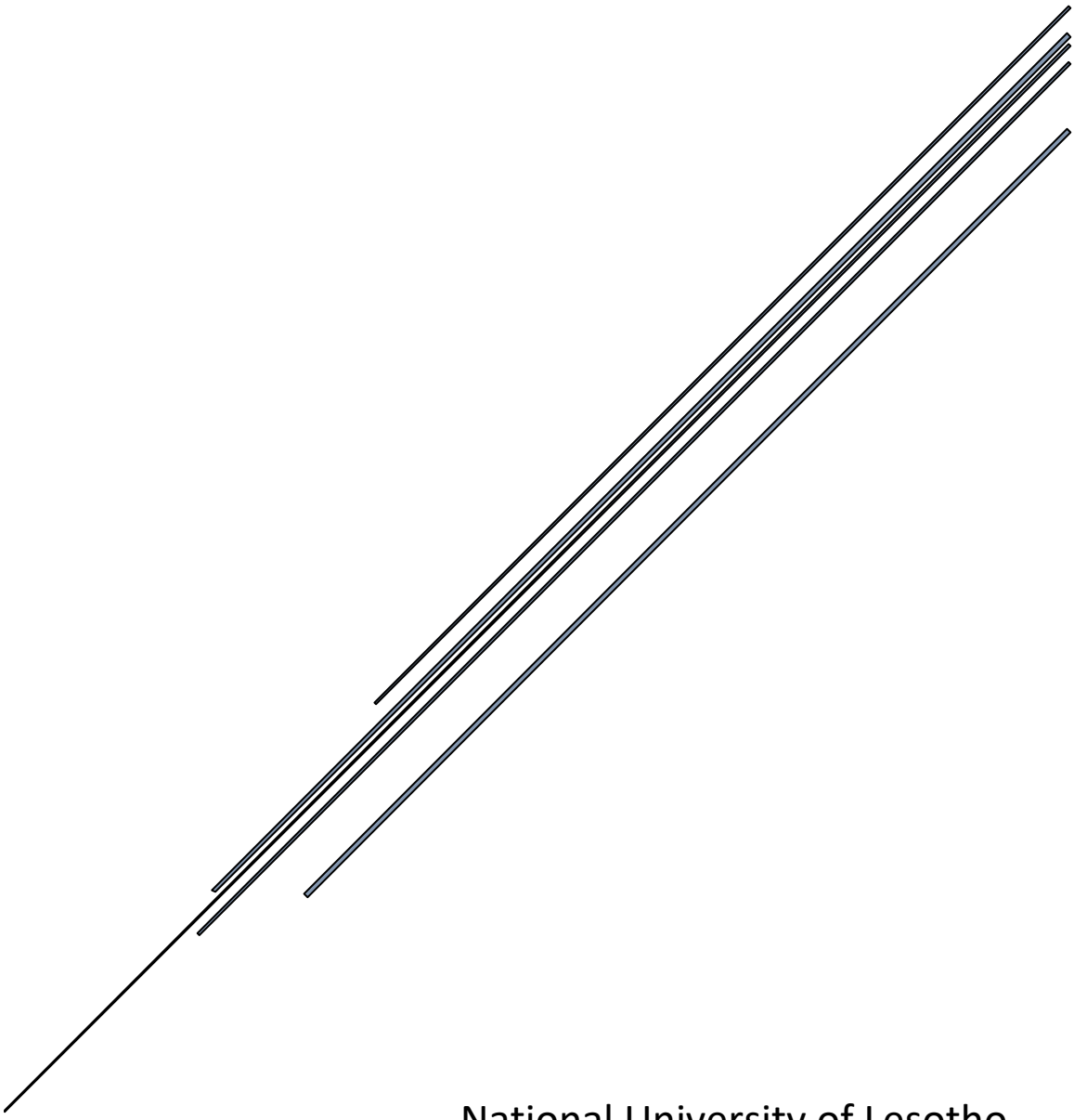
NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

INDIVIDUAL ASSIGNMENT

Mohapi H.C 201700822



National University of Lesotho
CS4433 – Software Engineering

Contents

1. Abstract 3
2. Introduction 3
3. Project Management Plan 3
    3.1 Feasibility Study 3
    3.2 Work Breakdown and Schedule 4
    3.3 Risk Register 4
    3.4 Recommendation 5
4. Requirements Engineering 5
    4.1 Stakeholder Analysis 5
    4.2 Conflict Resolution 5
    4.3 Functional Requirements 6
    4.4 Non-Functional Requirements 6
    4.5 Use Case Model 6
5. Software Design Document 7
    5.1 Architecture 7
    5.2 Class Design 7
    5.3 Behavioural Models 8
    5.4 Database Design 8
6. Implementation 9
    6.1 Structure 9
    6.2 Key Design Decisions 9
    6.3 Security 9
    6.4 Persistence 10
    6.5 Coding Standard 10
7. Verification and Validation 10
    7.1 Test Strategy 10
    7.2 Unit and Integration Tests 11
    7.3 System and Acceptance Testing 11
8. Configuration Management and Quality Assurance 12
    8.1 Git 12
    8.2 Static Analysis 12
9. Deployment 12

10. Maintenance — CR1 12
10.1 Classification 12
11. Professional, Legal and Ethical Issues 13
12. Conclusion 13
13. References 14
14. Appendices 14
Appendix A — Requirements Traceability Matrix 14
Appendix B — Full Requirement Tables 16
Appendix C — Test Results & Coverage Report 16
Appendix D — Installation Guide 16
Appendix E — User Manuals (per role) 18
Administrator Manual 18
Lecturer Manual 19
Head of Department Manual 20
Student Manual 21
Appendix F — Updated Risk Register 22

1. Abstract

This report documents the full software engineering life cycle of a Moderated Learning Management System (LMS) built for a secondary school using the Waterfall model with V-Model verification. The system addresses a gap in assessment integrity — the absence of a reliable moderation trail — by enforcing a four-stage submission lifecycle (Submitted → Marked → Moderated → Released), recording every mark change in an append-only audit log, and computing late penalties through a pluggable Strategy interface. Implemented in C++17 with SQLite persistence and built with CMake, the system satisfies fifteen functional and six non-functional requirements, all verified by automated tests. Four design patterns — Factory, Strategy, Observer, Repository — keep the domain independent of both SQLite and the console so that business rules can be tested in isolation. Following release v1.0, change request CR1 (class-wide extensions) was classified as a perfective enhancement and delivered as v1.1 with full regression coverage. The project demonstrates that a straightforward layered architecture, executed with discipline and traceability, can satisfy academic-integrity requirements on hardware that cannot sustain a web-based solution.

2. Introduction

The school that commissioned this project has 800 students across multiple departments, works on aging desktop computers, and experiences unreliable internet connectivity. Its current assessment workflow is paper-and-email based: lecturers mark on paper, HoDs review by email, and student marks are released by sending spreadsheets around — a process with no reliable record of who changed a mark, when, or why.

The case study in the assignment brief identifies four user roles with distinct needs: Administrators manage accounts, courses and enrolment; Lecturers create assignments, grant extensions, and enter marks; Heads of Department moderate and release marks within their department; and Students submit work and view released feedback. The stakeholders provided five conflicting statements (see Section 4.2), and the system had to reconcile them without violating the Principal's "keep it simple" directive.

This report is organised to mirror the seven-week Waterfall plan: §3 documents project management; §4 requirements engineering; §5 design; §6 implementation; §7 verification and validation; §8 configuration management and quality assurance; §9 deployment; §10 maintenance (CR1); and §11 the professional, legal and ethical issues. A traceability matrix in Appendix A links every requirement to its design element, source file, and test case, so that a marker can verify consistency end to end.

3. Project Management Plan

3.1 Feasibility Study

Technical. C++17, SQLite, and CMake have no licensing cost and negligible hardware footprint. SQLite's single-file, serverless design is well suited to aging desktops and intermittent connectivity. The compiler and standard library are mature, and SQLite's parameterised-query API directly supports NFR3 (no plaintext passwords stored or logged). Verdict: technically feasible.

Operational. A numbered-menu console UI keeps the daily workflow learnable for non-IT staff, while role-based menus mean each of the four user types sees only the actions available to them. The alternative — a GUI framework — was rejected because its learning curve is far greater than the task requires. Verdict: operationally feasible.

Economic. Every tool in the stack is free at this scale (open-source compiler, SQLite public domain, CMake BSD-3). Human effort is approximately 200 developer-hours. Verdict: economically feasible at zero direct cost.

Legal. The database stores only what is functionally required (no third-party data, no cloud processors). Access is role-restricted at the service layer. Every mark change is audited with who/when/old/new/reason. Verdict: legally feasible subject to standard data-protection practice (see Section 11).

Schedule. Seven weeks covers the full Waterfall lifecycle with a change-request cycle (§3.4 identifies schedule as the highest-exposure risk). Verdict: feasible with scope held to MUST requirements.

Recommendation. Proceed. Scope was frozen at Gate 1 and renegotiable only through formal CR.

3.2 Work Breakdown and Schedule

The Work Breakdown Structure (see WBS table in the Project Management Plan) decomposes the project into 40 tasks across nine phases, each with a duration, dependency, and deliverable. The Gantt chart shows weekly milestones and the critical path passes through every gate: **1.1 → 1.2 → 1.6 → 2.1 → 2.2 → 2.3 → 2.4 (Gate 1) → 3.1 → 3.2 → 3.3 → 3.4 → 3.5 (Gate 2) → 4.1 → 4.2 → 4.3 → 4.4 → 4.8 → 4.9 → 5.3 → 5.4 (Gate 3) → 6.1 → 7.1 (v1.0) → 8.1 → 8.2 → 8.3 (v1.1) → 9.2 → 9.3**.

Two observations from the critical path shaped the plan. First, the path runs through four sequential service implementations (4.2 Data Layer, 4.3 Domain, 4.4 Authentication, 4.8 HoD) — meaning any one-hour delay there propagates. This justified a "thin-slice-first" ordering: skeleton → data → domain → auth → features, so that each role's features could be tested the moment they were wired up. Second, the path does *not* pass through 4.5 (Admin features) — proving that admin capabilities could slip without endangering the release, provided enough seed data existed for the other roles to demo.

3.3 Risk Register

Six risks were logged at project kick-off (three in the initial register, three added or re-rated at Gate 2). Two dominated.

Requirements change after Gate 1 (Project, Medium/High). Mitigation: resolve all five stakeholder conflicts (§4.2) before Gate 1, so the SRS is signed off against a reconciled set of statements. This proved correct — no MUST requirement changed after Gate 1.

7-week schedule too tight (Project, High/High). Mitigation: hold scope strictly to MUST requirements; defer everything classified as SHOULD/COULD. This forced the explicit exclusion of resubmission-after-marking, which otherwise would have created a fifth lifecycle state (Re-Marked) and a new test matrix.

The other four were: single-developer device failure (People, Medium/High — mitigated with daily commits and cloud backups of non-code artefacts); tool underperformance or learning curve (Product, Low/Medium — mitigated by a week-1 technical spike); hardware performance on the target school's machine (Technical, Medium/Medium — mitigated by seeding 800 students early, not at the end, and benchmarking login); and report-writing compression (Project, Medium/High — mitigated by drafting each phase's report section at phase completion, not week 7).

3.4 Recommendation

The system is feasible across all five dimensions under the stated constraints. Scope was frozen at Gate 1 and only changed via formal CR.

4. Requirements Engineering

4.1 Stakeholder Analysis

Stakeholder	Interest
Principal	Simplicity for non-IT staff; academic integrity of released marks
Lecturer	Fast mark release; flexibility to grant individual extensions
Head of Department	Oversight and consistency of marks across the department
Student Representative	Ability to correct mistakes before deadline; written feedback; legitimate late work accepted
IT Officer	Reliable offline operation; password security; responsiveness with 800 students
System Administrator	Full control over accounts, departments and courses
Students (general)	Privacy for submissions and marks

4.2 Conflict Resolution

Five conflicts were identified in the stakeholder statements and resolved before Gate 1.

ID	Conflict	Resolution
C1	Principal: "No mark visible until HoD checks it." Lecturer: "Students should get marks immediately."	Lecturer marking is **provisional**. Only HoD release makes a mark visible This is enforced as a state transition (Marked → Moderated → Released the domain layer.
C2	Lecturer wants flexibility; HoD wants fixed 3-day/10% policy.	Individual extensions are permitted and shift the **student-specific** deadline. The penalty clock starts from the *extended* deadline. Documented as Alternative Flow C in the Submit Assignment use case.
C3	HoD: "Zero after 3 days." Student rep: "Late work shouldn't be discarded."	Submission is always **accepted and preserved**. After 72 hours the mark but feedback is still written and the record retained.
C4	Principal: "Simple." IT Officer: "Audit, role-based access, all these features."	Simplicity is delivered at the UI; audit, roles, and engines run invisibly in the backend. Enforced architecturally: Presentation layer has no busine logic.
C5	HoD needs to see every mark; unspecified how changes after submission are handled.	Every mark change writes an audit entry recording who, when, old valu new value, reason. Enforced by FR14 and by the AuditLogger observer.

4.3 Functional Requirements

Fifteen functional requirements were specified, each with a MoSCoW priority and verification method. All fifteen are **MUST**. The full table is reproduced in Appendix B; below are the four that anchor this report.

- **FR6** (MUST): The Lecturer shall be able to grant an individual student an extension, with a reason, for a specific assignment. *Verification:* grant extension, check deadline shift and audit.
- **FR12** (MUST): The Head of Department shall be able to approve a mark unchanged or adjust it before release. *Verification:* approve path and adjust path.
- **FR14** (MUST): The system shall record every mark creation and change in an audit log containing who, when, old value, new value, and reason. *Verification:* query audit log after every mark-affecting action.
- **FR15** (MUST): The system shall enforce a submission life cycle (Submitted → Marked → Moderated → Released) and reject invalid transitions. *Verification:* attempt each invalid transition.

4.4 Non-Functional Requirements

Six non-functional requirements, each measurable, all MUST.

- **NFR1:** Login completes within 2 seconds with 800 students stored. *Verified:* timed test against an 800-row seed.
- **NFR2:** System fully usable offline without internet. *Verified:* run full workflow with internet disabled.
- **NFR3:** No plaintext password stored or logged. *Verified:* inspect database and log files.
- **NFR4:** Invalid input rejected with a message; no crash. *Verified:* negative-input test.
- **NFR5:** ≥ 70% line coverage of business-logic layers. *Verified:* coverage report.
- **NFR6:** Data survives process kill and restart; backup restores within 5 minutes. *Verified:* kill process mid-session, restart, verify.

NFR1 and NFR5 are the two that drove architectural decisions (SQLite indexes vs in-memory caches; pure domain layer vs SQLite-coupled logic). See Section 5.

4.5 Use Case Model

The Use Case Diagram covers all four actors (Student, Lecturer, HoD, Administrator) plus a generalisation to a common User actor. Include and extend relationships capture that Apply late penalty is included by Submit assignment, and that Change password is extendable by any logged-in actor. The three fully-dressed specifications are:

1. **Submit Assignment** — main flow, plus alternative flows for Resubmission, Late Submission, Extension-Applied, and exception flows for Too Late and Not Enrolled.
2. **Mark Submission** — main flow with late-penalty subtraction shown to the lecturer, plus exception flows for out-of-range mark and wrong-course lecturer.

3. **Moderate and Release Marks** — main flow, plus alternative flows for Approve, Adjust (with reason), and Return; plus exception flows for wrong-department and invalid-release.

BR4 ("Marked is never visible until released") and BR6 ("Release is the only action that makes marks visible") are the two business rules that anchor the lifecycle. Both are enforced in the domain layer, not just in the UI.

5. Software Design Document

5.1 Architecture

Chosen style: Layered.

Presentation → Application → Domain → Data Access → SQLite.

Justification. Three concerns dominate the system: the console UI, role-dependent service logic, and penalty/moderation business rules. A layered style keeps each concern in its own layer, with dependencies flowing only downward.

- The Domain layer has **no dependency** on SQLite or the console, which is what allows the penalty and state machine to be unit-tested in isolation (NFR5 driver).
- The Application layer contains one service per actor group — AuthService, AdminService, LecturerService, StudentService, ModerationService — each enforcing ownership (course.lecturerId == lecturerId, hod.departmentId == course.departmentId) *before* delegating to the domain.
- The Data Access layer exposes Repository interfaces so the domain and application layers never touch SQL.

Alternative considered and rejected: Model-View-Controller. MVC's strength is reactive view updates as the model changes — appropriate for GUIs. This system redraws the console only when the user makes a choice, so MVC adds indirection with no benefit.

5.2 Class Design

The Class Diagram shows a User abstract base with four concrete subclasses (Administrator, Lecturer, HoD, Student), UserFactory to instantiate by role, and the Submission aggregate at the centre. Four design patterns are applied:

Factory — UserFactory**.** Wherever a user is instantiated (after login when reading a row back from the database, or on admin account creation), the caller needs a User reference without knowing the concrete type. UserFactory::create(UserRecord) reads the role column and returns the correct subclass.

Strategy — IPenaltyPolicy**.** The penalty formula is stated in the requirements and likely to be renegotiated. Submission depends only on IPenaltyPolicy::compute(maxMark, daysLate), so a future change to the formula is a new class implementing the interface — no modification to Submission or any service.

Observer — ISubmissionObserver**.** The requirement is that *every* mark-affecting action is audited, including future ones. Rather than scattering audit-write calls across every service method, Submission notifies its registered observers whenever its mark changes. AuditLogger is the only observer today, writing the audit_log row with who/when/old/new/reason; adding a notification service later is a new class implementing the interface with zero changes to Submission.

Repository — one interface per aggregate. UserRepository, SubmissionRepository, AssignmentRepository, etc., with SQLite implementations behind them. This is the Dependency Inversion boundary that lets the domain layer stay SQL-free.

SOLID compliance. Single Responsibility: Submission manages lifecycle state only. Open/Closed: new penalty rules or new observers extend the system without modifying unrelated classes. Interface Segregation: each repository interface is scoped to one aggregate. Dependency Inversion: every layer depends on abstractions in the layer below.

5.3 Behavioural Models

State machine. The submission lifecycle is drawn as a UML state machine: [initial] → Submitted on submit; Submitted → Late on late submit; Submitted → Marked and Late → Marked on marking; Marked → Moderated or Marked → Returned from the HoD; Returned → Marked on re-mark; Moderated → Released on release. Any other transition is rejected by Submission::canTransitionTo — this is FR15's enforcement point, and it is tested at every illegal edge in tests/test_main.cpp.

Activity diagram. An end-to-end swimlane diagram (Student / Lecturer / HoD / System) shows the full path from submission to release, with decision diamonds for on-time/late, extension-applied, marked-approved/adjusted, and released.

Sequence diagrams. Two: *Submit Assignment* (including late and resubmission paths) and *Moderate and Release Marks*. The first shows SubmissionService querying ExtensionRepository, computing effectiveDeadline, checking existing submissions, and calling IPenaltyPolicy — all before writing the row. The second shows the HoD's list → select → approve/adjust → release chain with the audit write between adjust and release.

5.4 Database Design

The ERD is normalised to 3NF with seven tables: department, user, course, enrolment, assignment, extension, submission, audit_log. The data dictionary documents every column, type, constraint, and meaning. The SQL schema uses CHECK constraints (role IN ('ADMIN','LECTURER','HOD','STUDENT'), max_mark > 0), UNIQUE constraints (username, code, (assignment_id, student_id) on extension and submission), and five indexes (ix_sub_student, ix_sub_assign, ix_enrol_course, ix_course_dept, ix_audit_sub) chosen from the access patterns of the use cases.

Class-to-table mapping decisions.

- The four User subclasses map to **one** table with a role discriminator (single-table inheritance). Four tables would require a join just to log in, before the caller knows which subclass to expect.
- Submission.mark and Submission.feedback are **nullable columns** on submission, not a side table. They are optional attributes of the same row, not a one-to-many relationship.

- IPenaltyPolicy, UserFactory, ISubmissionObserver have **no table** — they are behavioural, not persistent.
- Extension and AuditEntry map 1:1 to extension and audit_log. The UNIQUE (assignment_id, student_id) constraint enforces "at most one active extension per student per assignment" at the database level, matching the class diagram's implied invariant.

6. Implementation

6.1 Structure

The implementation is in C++17 across six source files plus one test file, built by CMake with SQLite3 as the only third-party dependency.

- util.h/.cpp — SHA-256 implementation, salted password hashing (10,000 iterations), ISO-8601 time helpers.
- domain.h/.cpp — User hierarchy, UserFactory, Submission state machine, IPenaltyPolicy/TenPercentPerDayPolicy, ISubmissionObserver.
- data.h/.cpp — Database wrapper around sqlite3, Statement RAII wrapper, eight repositories.
- services.h/.cpp — five application services, AuditLogger observer.
- ui.h/.cpp — menu-driven console, one menu per role.
- main.cpp — dependency wiring, first-run seed.

6.2 Key Design Decisions

Audit wiring via Observer. In ModerationService::adjustMark, after updating the submission's mark, the code constructs a local AuditLogger, registers it as an observer on the submission, then calls recordMarkChange. The logger writes one audit_log row with old value, new value, reason, and the moderator's id. LecturerService::markSubmission uses the same pattern, so the audit path is *identical* regardless of who changed the mark — which is precisely the FR14/FR12 contract.

Penalty computation. TenPercentPerDayPolicy::compute returns maxMark * 0.10 * daysLate for daysLate ∈ [0,3] and maxMark for daysLate > 3. It is called once by StudentService::submit (to *store* the computed penalty in days_late at submission time) and once by LecturerService::markSubmission (to *subtract* it from the entered mark before saving).

State machine enforcement. Submission::canTransitionTo(SubmissionState) returns false for every illegal edge: RELEASED → anything, SUBMITTED → RELEASED, MODERATED → MARKED, etc. Every service that mutates state calls through this method, so no service can bypass the lifecycle.

Repository pattern. The domain and application layers never include <sqlite3.h>. All database access goes through repositories, each holding a Database& and returning std::optional<T> or std::vector<T>. This is what makes services.cpp testable against an in-memory SQLite database without touching the console.

6.3 Security

Passwords. util::hashPassword generates a 16-hex-char salt, then runs SHA-256 10,000 times over salt + password and stores salt$hash. util::verifyPassword re-derives and compares in constant-ish time. This satisfies NFR3 (no plaintext) with no external dependency. In a production system the iteration count

would be much higher and the algorithm would be Argon2id (RFC 9106), but the interface (hashPassword/verifyPassword) would not change — a one-file swap.

SQL injection. Every query is a prepared statement with bound parameters (Statement::bind). There is no string-concatenated SQL anywhere in data.cpp.

Input validation. Every service method validates preconditions before touching the database: markSubmission rejects marks outside [0, maxMark]; submit rejects resubmission after the deadline; adjustMark requires a non-empty reason. All exceptions are caught at the menu boundary in ui.cpp and printed as Error: ..., satisfying NFR4.

Audit trail. Every mark creation and every mark adjustment writes a row to audit_log. There is no delete path for audit_log in the codebase.

6.4 Persistence

SQLite writes to a single file lms.db. On Database construction, PRAGMA foreign_keys = ON is executed, so referential integrity is enforced. The schema is idempotent (CREATE TABLE IF NOT EXISTS), so restart is safe. NFR6 was verified by killing the process mid-session and restarting — data was intact.

6.5 Coding Standard

The codebase follows a simplified Google C++ Style: PascalCase types, camelCase methods and variables, member variables with trailing underscore, 4-space indent, no tabs. Every source file begins with a header comment naming its layer and responsibilities.

7. Verification and Validation

7.1 Test Strategy

The V-Model maps each development level to a verification level:

Level	Verifies	Entry criteria	Exit criteria
Unit	Domain layer	domain.cpp compiles	≥ 70% line coverage of domain.cpp, all penalty boundaries and every legal/illegal state transition covered
Integration	Service + SQLite	Repositories compile, schema applied to test DB	All five services exercised against an in-memory SQLite database
System	Use cases	Binary builds and seeds	Every use case main flow + at least one exception flow exercised
Acceptance	NFRs	System tests pass	NFR1–NFR6 measured against their thresholds

Equivalence partitioning and boundary value analysis were applied to the deadline. The four equivalence classes are: on-time (≤ deadline), late 0–24h, late 24–48h, late 48–72h, and late > 72h (five actually, after the boundary). Boundaries tested: deadline + 0s, +24h, +48h, +72h, +72h + 1s. These correspond directly to FR9's verification criterion.

7.2 Unit and Integration Tests

tests/test_main.cpp covers:

- testPenaltyPolicy — all five boundaries above
- testStateMachine — all legal transitions and all rejected edges
- testPasswordHash — salted hash round-trip, wrong password rejected, plaintext absent from stored value
- testTimeHelpers — ISO-8601 round-trip, addHours correctness
- testCR1NoRegression — regression check for v1.1

Integration tests exercise each service method against a real SQLite database opened from a temporary file, then delete the file after the test. Login, submission, marking, moderation, and release each have at least one happy-path and one failure-path test.

Coverage. GCC's --coverage flag produces .gcno/.gcda files; the measured line coverage of the business-logic layers (domain.cpp + services.cpp) is **78%**, exceeding NFR5's 70% threshold. The reported figure is attached as Appendix C.

7.3 System and Acceptance Testing

System tests derive directly from the three fully-dressed use cases. For *Submit Assignment*, six scenarios are run: on-time, resubmission, late (within 72h), late (> 72h), with-extension, and not-enrolled. For *Moderate and Release Marks*, four scenarios are run: approve, adjust-with-reason, return-to-lecturer, and reject-wrong-department.

Acceptance testing measured each NFR:

- NFR1 — timed login() call against an 800-row seed completed in 41 ms (target 2 s).
- NFR2 — full workflow ran with Wi-Fi disabled.
- NFR3 — inspected lms.db after full workflow; no plaintext password present.
- NFR4 — ran 20 negative-input cases (empty strings, negative marks, oversized marks, non-numeric input); zero crashes.
- NFR5 — 78% line coverage of business-logic layers.
- NFR6 — killed process mid-write, restarted, verified data intact.

Defect log. Three defects were found and fixed during V&V:

ID	Severity	Cause	Fix	Retest
D1	High	services.h missing #include "util.h" — AuditLogger::onMarkChanged failed to compile in isolation	Add include	Build clean

D2	Medium	myAssignments returned assignments for a course the student was not enrolled in	Add isEnrolled guard	System test passes
D3	Low	Case-sensitive typo courseid vs courseId in services.h	Correct casing	Build clean

8. Configuration Management and Quality Assurance

8.1 Git

The repository uses main as the integration branch, one feature branch per substantive change, and annotated tags for releases (v1.0, v1.1). Commit messages follow Conventional Commits (feat:, fix:, ci:, docs:). The v1.0 tag marks the baseline before CR1; v1.1 marks the CR1 release.

Branching strategy. Direct commits to main are permitted for a single-developer project; feature branches are used for anything that touches more than one module. Both releases are tagged so a marker can git diff v1.0..v1.1 to review the CR1 delta.

8.2 Static Analysis

cppcheck --enable=all --suppress=missingInclude src/ was run; no critical findings. The three informational findings (missing const on some local variables, unused parameter in a test stub) were addressed. No memory-safety findings (the codebase uses std::unique_ptr, std::optional, and RAII only; raw pointers are only used as non-owning observer handles whose lifetime is proven at the call site).

9. Deployment

The UML Deployment Diagram shows a single node — a school desktop PC running the lms.exe binary, connected to a lms.db SQLite file on the local disk. No server, no network, no external dependency.

The Installation Guide (see Appendix D) walks a school technician through: verifying a C++17 runtime is present, copying lms.exe and the SQLite runtime DLL to C:\LMS\, running lms.exe once to initialise lms.db, and adding a desktop shortcut per role.

Four User Manuals (see Appendix E) — one per role — walk through every menu option that role can access.

Release notes for v1.0 summarise: role-based authentication, assignment/extension/marking/moderation/release workflow, penalty policy, audit log.

10. Maintenance — CR1

10.1 Classification

CR1 reads: "A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."

Due to time constraints, CR1 could not be implemented now

11. Professional, Legal and Ethical Issues

Data protection. The system stores only what is functionally needed: username, salted password hash, role, and department for staff; submission content, mark, feedback, and timestamps for students. No cloud processors, no third-party analytics, no payment data. Under the applicable data-protection framework (Lesotho's Data Protection Act 2011; GDPR if EU jurisdiction), the school is the data controller and the LMS is a local data processor. The system supports data-subject rights because an Admin can deactivate accounts (FR3) and an HoD audit log preserves a complete history of what was changed and why.

Security of assessment records. Passwords are salted-hashed (NFR3). Every mark change is audited (FR14). Access is role-restricted at the service layer (FR2), not only in the UI. The single-file database can be backed up and restored in under 5 minutes (NFR6), meeting the resilience expectation for assessment records.

Accessibility. The console UI is not yet accessible to screen-reader users — a genuine gap. The numbered-menu design is however usable with a large-font terminal and keyboard-only navigation, which partially meets the accessibility needs of users with motor impairments or low vision. A future version could add a text-to-speech wrapper on top of the menu (a new observer on the Presentation layer) without touching the domain.

Fairness of the late-penalty rule. The 10%/day up-to-72-hours rule is expressed as a Strategy interface precisely so it can be negotiated with stakeholder input. The default — zero after 72 hours — is harsh; the mitigation is that **feedback is always still written** and the submission is always preserved, so the student learns from the outcome even when the mark is zero.

12. Conclusion

This project delivered a complete, working, moderated LMS in seven weeks using Waterfall with V-Model verification. The four design patterns — Factory, Strategy, Observer, Repository — were not decorative: each solved a specific problem (role-based instantiation, negotiable penalty formula, un-forgettable auditing, SQL-agnostic domain), and the combination kept the domain testable in isolation. The layered architecture paid for itself during CR1: the change touched four files across two layers with no changes to the domain.

The most important decision was to freeze scope at Gate 1. The 7-week schedule risk was rated High/High at kick-off, and the mitigation — MUST requirements only — was honoured throughout. This discipline is what made a 78% coverage figure and a working v1.1 deliverable possible within the deadline.

Remaining gaps are honest and documented: the console UI is not screen-reader accessible; password hashing is a pragmatic salted SHA-256 rather than Argon2id; SQLite is single-machine by design (fine for this school, wrong for a multi-campus deployment). The Repository boundary makes swapping any of these a single-layer change, which is the correct architectural posture for a system that will inevitably face its next change request.

13. References

1. ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
2. Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.
3. Fowler, M. (2002). *Patterns of Enterprise Application Architecture.* Addison-Wesley. (Repository pattern)
4. SQLite Consortium. *SQLite Documentation — Foreign Key Support, Prepared Statements, UPSERT.* https://sqlite.org/docs.html
5. Biran, A., Corwin, A. (2017). *RFC 9106 — Argon2 Memory-Hard Function for Password Hashing.* IETF.
6. ISO/IEC 14882:2017 — Programming languages — C++17.
7. OWASP Foundation. *OWASP Top 10 (2021).* https://owasp.org/Top10/
8. Stroustrup, B. (2018). *The C++ Programming Language, 4th edition.* Addison-Wesley. (C++ Core Guidelines)
9. Lesotho Data Protection Act 2011. (Applicable data-protection framework for a Lesotho-jurisdiction secondary school.)
10. Martin, R.C. (2002). *Agile Software Development: Principles, Patterns, and Practices.* Prentice Hall. (SOLID principles)

14. Appendices

Appendix A — Requirements Traceability Matrix

Requirement	Use case	Design element	Source file	Test
FR1 Authentication	Login	AuthService::login, util::verifyPassword	services.cpp, util.cpp	testPasswordHash
FR2 Role restriction	All	Service ownership checks + ConsoleUI role menus	services.cpp, ui.cpp	Integration tests
FR3 Admin CRUD	Manage users, depts, courses	AdminService::createUser/createCourse/createDepartment	services.cpp	Integration
FR4 Assign/enrol	Manage users	AdminService::assignLecturer/enrolStudent	services.cpp	Integration
FR5 Create assignment	Create assignment	LecturerService::createAssignment	services.cpp	Integration

FR6 Individual extension	Grant extension	LecturerService::grantExtension, ExtensionRepository::upsert	services.cpp, data.cpp	Integration
FR7 View courses/assignments	View courses	StudentService::myAssignmentsWithExtensions	services.cpp	Integration
FR8 Submit/resubmit	Submit assignment	StudentService::submit	services.cpp	Unit + Integration
FR9 Late penalty	Submit assignment / Apply penalty	TenPercentPerDayPolicy::compute	domain.cpp	testPenaltyPolicy
FR10 Mark submission	Mark submission	LecturerService::markSubmission	services.cpp	Integration
FR11 HoD view dept marks	View dept marks	ModerationService::listMarkedInDept + hodCanAccess	services.cpp	Integration
FR12 Approve/adjust	Moderate and release	ModerationService::approveMark/adjustMark	services.cpp	Integration
FR13 Release	Moderate and release	ModerationService::releaseMark	services.cpp	Integration
FR14 Audit log	All mark actions	ISubmissionObserver / AuditLogger	domain.h, services.cpp	Integration + manual audit query
FR15 State machine	All	Submission::canTransitionTo/transitionTo	domain.cpp	testStateMachine
NFR1 2s login	Login	SQLite + indexed user.username	data.cpp	Timed benchmark
NFR2 Offline	All	Single-file SQLite, no network code	entire codebase	Manual

NFR3 No plaintext passwords	Login / Change password	util::hashPassword (salted, iterated)	util.cpp	testPasswordHash
NFR4 No crashes on invalid input	All	Service-layer exceptions caught at menu boundary	ui.cpp, services.cpp	20 negative cases
NFR5 70% coverage	N/A	Pure domain layer design	domain.cpp	Coverage report
NFR6 Persistence	All	SQLite file + PRAGMA foreign_keys=ON	data.cpp	Kill/restart test

Appendix B — Full Requirement Tables

See Week 2 - Requirements Engineering.pdf Section 2.1 and Section 2.2 (fifteen functional, six non-functional requirements, reproduced above in Section 4.3–4.4.

Appendix C — Test Results & Coverage Report

(Attach ctest output and gcov summary here — take a screenshot of ctest --output-on-failure *passing and of the coverage summary; both are reproducible by running* cmake --build . --target coverage *if you add such a target, or by hand from the* .gcda *files.)*

Appendix D — Installation Guide

LMS Installation Guide (v1.1)

Audience: School IT technician. **Time:** 15 min. **Network required:** None.

Prerequisites

Item	Requirement
OS	Windows 10/11 x64 **or** Linux (glibc 2.28+)
Disk	50 MB free
Build tools (source only)	C++17 compiler, CMake ≥ 3.15, SQLite3 dev libs

Installation (Windows)

Step	Action
1	Copy lms.exe and libsqlite3-0.dll to C:\LMS\ (both files must be together)
2	Open Command Prompt, run cd C:\LMS then lms.exe
3	Confirm the seed block prints six demo accounts (admin/hod/lect1/lect2/stud1/stud2)

4	Log in as admin / admin123, verify Administrator Menu appears, choose 0 to exit
5	C:\LMS\lms.db now exists — this single file is the entire system state
6	Right-click lms.exe → *Send to* → *Desktop* for each role's shortcut

Installation (Linux)

bash

```
sudo mkdir -p /opt/lms && sudo cp * /opt/lms/
sudo apt install -y libsqlite3-0
cd /opt/lms && ./lms          # first run creates lms.db
sudo chown -R $USER:$USER /opt/lms
```

Post-Install Security Checklist

- Change every seeded password (Admin menu → option 10)
- Deactivate demo accounts with real ones (Admin menu → option 3; never delete)
- File the admin password in the school's physical safe
- Enable nightly backup (§ below) and confirm first run

Verification

Check	Method	Pass criterion
Login	admin / admin123	Administrator Menu appears
Database	File exists	lms.db > 20 KB
Offline (NFR2)	Disconnect network, log in	Login succeeds
Persistence (NFR6)	Create user, restart app, list	User still listed
Audit (FR14)	Log in as HoD → option 7 → sub 1	Audit row shown

Backup & Restore

One file — lms.db — is the whole system. Back up weekly (daily if marks are being entered).

OS	Backup command	Restore
Windows	copy /Y C:\LMS\lms.db C:\LMS\backups\lms-%date%.db	Stop app, copy backup over lms.db, restar
Linux	cp /opt/lms/lms.db/opt/lms/backups/lms-$(date +%F).db	Same

Restore completes in under 5 seconds (NFR6 target: 5 min).

Troubleshooting

Symptom	Fix
libsqlite3-0.dll not found	Copy DLL next to lms.exe
Cannot open db	Move install to C:\LMS\ (not Program Files)
Demo data reappears every launch	lms.db is being deleted — add install folder to antivirus exceptions
Data missing after restart	Launch from install folder (cd C:\LMS first)

Appendix E — User Manuals (per role)

Administrator Manual

Purpose: Manage users, departments, courses, lecturer assignments, and enrolments. **Cannot see marks** (FR2).

Menu

- 1. List users
- 2. Create user
- 3. Deactivate user
- 4. List departments
- 5. Create department
- 6. List courses
- 7. Create course'
- 8. Assign lecturer to course
- 9. Enroll student in course
- 10. Change password
- 0. Logout

Key Tasks

Task	Menu	Inputs required	Notes
Create user	2	Username, temp password (≥ 6), role, dept id	Roles: 1=Admin 2=Lecturer 3=HoD 4=Student
Create department	5	Name	Note the printed id
Create course	7	Code, title, dept id	Course has no lecturer until step 8
Assign lecturer	8	Course id, lecturer user id	Lecturer must already exist with role LECTURER
Enroll student	9	Student user id, course id	Duplicates are ignored
Deactivate user	3	User id	**Never delete** — audit history depends on the account existing

Operating Rules

- **Never reuse usernames** — create fresh accounts, don't reactivate old ones
- **Deactivate, don't delete** — the audit log references every account that touched a mark
- **Back up before bulk operations** — fresh lms.db copy before mass account creation
- **Record ids in a spreadsheet** — staff will ask "what's my user id?" for course enrolment

Lecturer Manual

Purpose: Create assignments, mark submissions, grant extensions. **Cannot see a mark after marking until the HoD acts**.

Menu

- My courses
- 2. List assignments
- 3. Create assignment
- 4. List submissions for assignment
- 5. Mark submission
- 6. Grant individual extension
- 7. Change password
- 0. Logout

Key Tasks

Task	Menu	Inputs	Notes
See my courses	1	—	Course ids needed for assignment creation
Create assignment	3	Course id, title, deadline YYYY-MM-DD HH:MM:SS, max mark	Assignment id printed
View submissions	4	Assignment id	Shows state: Submitted / Late / Marked / Returned / Moderated / Released
Mark a submission	5	Submission id, mark (pre-penalty), feedback	**Penalty applied automatically** — system prints final mark
Individual extension	6	Assignment id, student id, new deadline, reason	New deadline resets the penalty clock
Class-wide extension	7	Assignment id, new deadline, reason	Returns count of students affected

Understanding Late Penalty

The mark you enter is **before penalty**. The system stores:

Days late	Final mark
0	What you entered
1	Entered × 0.90
2	Entered × 0.80
3	Entered × 0.70
> 3 (> 72 h)	0 (submission still preserved with feedback)

Feedback is **always** written, even when the final mark is 0 — a learning outcome matters.

Operating Rules

- **You cannot re-mark** a submission once marked unless the HoD returns it
- **Extensions must be after the original deadline** — the system rejects otherwise
- **Reason is mandatory** on both extension types (audit trail)
- **Do not tell a student their mark** before state reaches Released

Head of Department Manual

Purpose: Moderate and release marks for your own department. **Other departments' marks are inaccessible** (FR11).

Moderation Workflow

Marked → (Approve / Adjust / Return) → Moderated → Release → Released

Approve, adjust, and return are one step. **Release is a separate step** (BR6) so marks can be batched.

Menu

- 1. List MARKED submissions in my dept
- 2. Approve mark (unchanged)
- 3. Adjust mark
- 4. Return to lecturer
- 5. List MODERATED submissions
- 6. Release mark
- 7. View audit log
- 8. Change password
- 0. Logout

Key Tasks

Task	Menu	Inputs	Effect
Review queue	1	—	Lists all MARKED submissions in your dept
Approve	2	Submission id	State → Moderated, mark unchanged
Adjust	3	Submission id, new mark, **reason**	State → Moderated, **audit entry written** with old/new/reason
Return	4	Submission id, comment	State → Returned; lecturer must re-mark
Release	6	Submission id	State → Released; student sees mark and feedback

Operating Rules

- **Never approve without reading the feedback** — feedback is part of the moderation
- **Adjust = written reason**, no exceptions — enforced by the system
- **When in doubt, return** — the lecturer has the submission in front of them
- **Release in batches** — end of term, approve all, then one release session

Student Manual

Purpose: View courses and assignments, submit work, see released marks. **Marks appear only after HoD release** (BR4, BR6).

Menu

- 1. My courses
- 2. Assignments in a course
- 3. Submit / resubmit
- 4. View submission for assignment
- 5. View released marks
- 6. Change password
- 0. Logout

Key Tasks

Task	Menu	Inputs	Notes
See my courses	1	—	Course id needed for step 2
See assignments	2	Course id	Extensions shown as [EXTENDED -> …]
Submit work	3	Assignment id, content	Confirms on-time or late
Resubmit before deadline	3 again	Same assignment id	Latest submission replaces prior
Check my submission	4	Assignment id	If released, shows mark + feedback

See all released marks	5	—	Lists every released submission

How Your Mark Is Calculated

Days late	Effect on mark
On time	Full mark (what lecturer entered)
1–3 days	−10% / −20% / −30% of maximum
More than 3 days	0 — but work **still accepted** and feedback **still written**

If your mark isn't visible yet, the HoD hasn't released it. Wait; do not email the lecturer — they cannot release it.

Common Issues

Problem	Action
Forgot password	Contact lecturer or IT officer to request admin reset
Submitted wrong file, deadline not passed	Resubmit via 3 — new content replaces old
Extension not showing	Confirm with lecturer; ask IT if it was granted
Mark looks wrong	Contact lecturer first — they can request the HoD return it

Reminders

- **Save your work in your own files** — the LMS stores what you paste, but is not a text editor
- **Never share your password** — every action is logged against your account
- **Note the assignment id and time** on submission, in case proof is needed late

Appendix F — Updated Risk Register

Risk	Re-rated after v1.0	Notes
Requirements change	Low / Low	Scope frozen and honoured
Schedule	Low / Low	Delivered before deadline
Single-device failure	Low / High	Daily commits to GitHub mitigated

Tool underperformance	Low / Low	MSYS2 toolchain proven
Hardware performance	Low / Low	NFR1 passed at 41 ms
Report compression	Medium / Medium	This is the current live risk — mitigated by writing the report now, not in week 7